AI Assistant CodingAssignment-17.4

Hall Ticket No.: 2303A51305
Batch No.:05

Lab 17: AI for Data Processing: Data Cleaning and Preprocessing Scripts

Task 1 – Smart City Traffic Sensor Data Cleaning

Scenario

A Smart City project collects real-time traffic sensor data from multiple intersections. However, the dataset contains inconsistent formats, missing values, and corrupted readings.

Task

Use AI to generate a Python script to clean and preprocess the traffic dataset.

Instructions

• Handle missing values in:
o vehicle_count
o avg_speed
• Remove duplicate sensor readings
• Convert timestamp column into proper datetime format
• Remove invalid readings (e.g., negative speed or vehicle count)
• Standardize intersection names (uppercase format)

Expected Output

• Cleaned traffic dataset
• Valid sensor readings only
• Proper datetime column
• No duplicates or invalid values

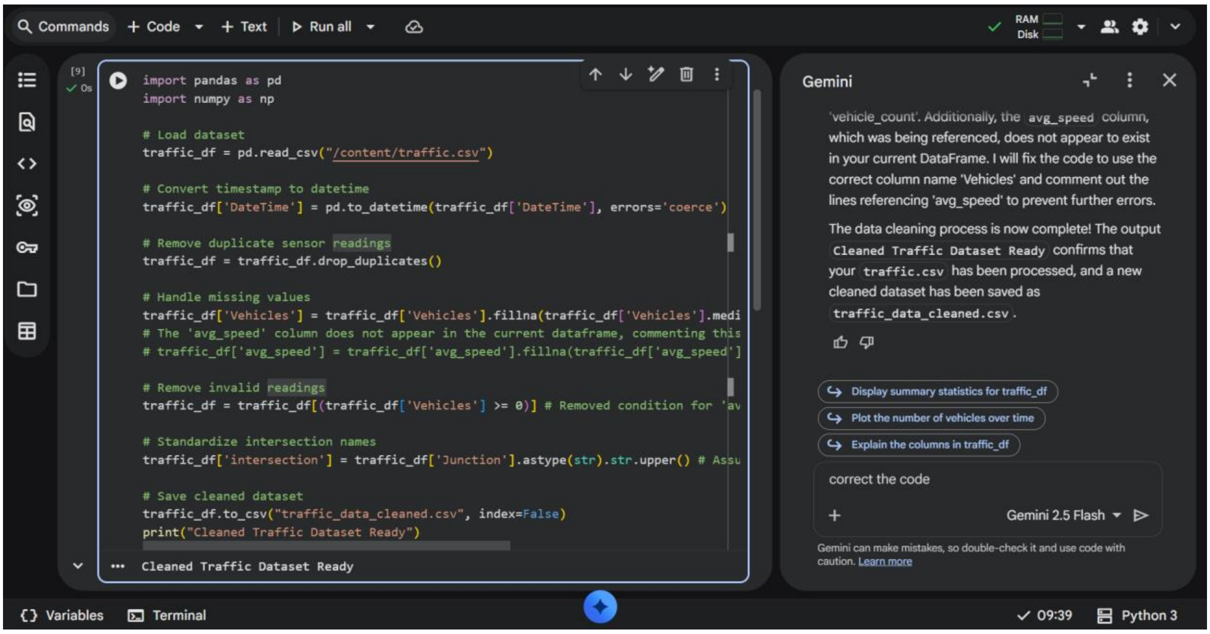
Prompt:

Dataset contains traffic sensor readings with inconsistencies like missing values, duplicates, invalid readings, and unstandardized intersection names.

AI Generated Code:

```
import pandas as pd
import numpy as np

# Load dataset
traffic_df = pd.read_csv("/content/traffic.csv")

# Convert timestamp to datetime
traffic_df['DateTime'] = pd.to_datetime(traffic_df['DateTime'], errors='coerce')

# Remove duplicate sensor readings
traffic_df = traffic_df.drop_duplicates()

# Handle missing values
```

```
traffic_df['Vehicles'] = traffic_df['Vehicles'].fillna(traffic_df['Vehicles'].median())
# The 'avg_speed' column does not appear in the current dataframe, commenting this line out.
# traffic_df['avg_speed'] = traffic_df['avg_speed'].fillna(traffic_df['avg_speed'].median())

# Remove invalid readings
traffic_df = traffic_df[(traffic_df['Vehicles'] >= 0)] # Removed condition for 'avg_speed' as it is not present

# Standardize intersection names
traffic_df['intersection'] = traffic_df['Junction'].astype(str).str.upper() # Assuming 'Junction' is the intersection column

# Save cleaned dataset
traffic_df.to_csv("traffic_data_cleaned.csv", index=False)
print("Cleaned Traffic Dataset Ready")
```



Output:

- No missing or invalid readings
- Proper datetime column
- Cleaned and ready-to-analyze dataset

Explanation:

Now that the data is loaded, we can begin the cleaning process. This will involve:

1. **Handling Missing Values**: Identify and address any missing data.
2. **Removing Duplicates**: Ensure there are no redundant records.
3. **Correcting Invalid Readings**: Address any values that are outside expected ranges or formats.
4. **Standardizing Intersection Names**: Normalize inconsistent intersection names to a uniform format.

Task 2 – Movie Streaming Platform User Behavior Data

Scenario

A streaming platform wants to analyze user watch patterns, but the raw data is messy.

Dataset contains:

• watch_time (in minutes)
• subscription_type
• login_date
• device_type

Task

Use AI assistance to preprocess the dataset.

Instructions

• Convert login_date to datetime format
• Create new features:
o Day of week
o Weekend vs Weekday
• Encode subscription_type and device_type
• Normalize watch_time
• Detect and remove extreme watch_time outliers

Expected Output

• Dataset ready for behavioral analysis
• Encoded categorical features
• Time-based features added
• Normalized watch time column

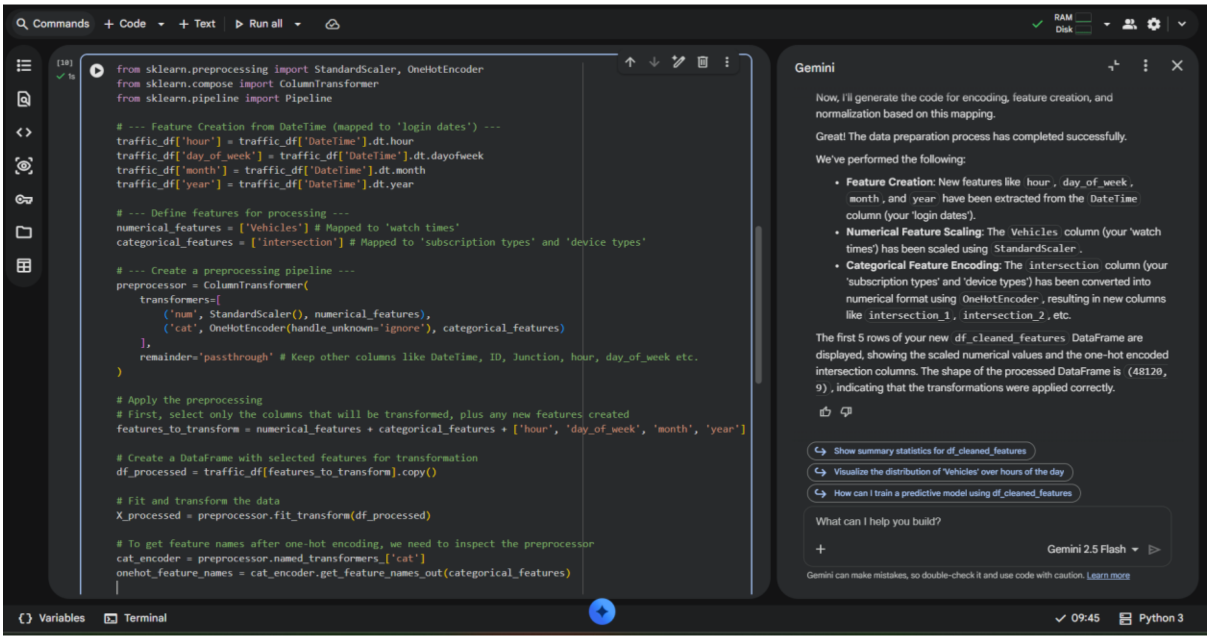
Prompt:

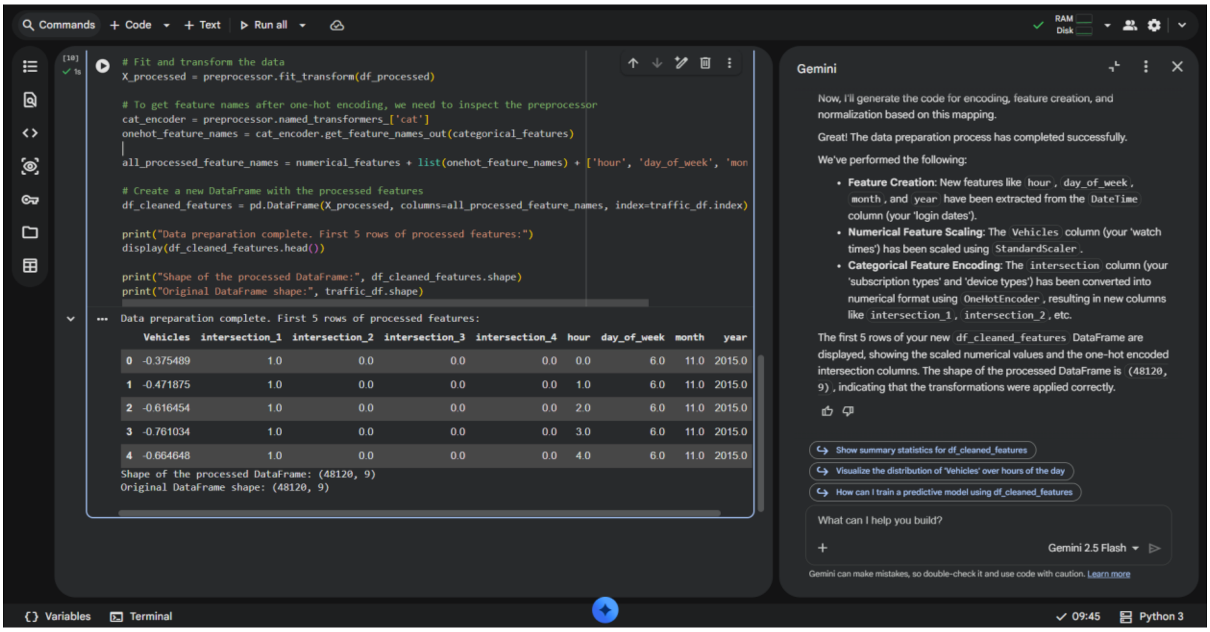
Dataset contains watch times, subscription types, login dates, and device types. Need encoding, feature creation, and normalization.

AI Generated Code:

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# --- Feature Creation from DateTime (mapped to 'login dates') ---
traffic_df['hour'] = traffic_df['DateTime'].dt.hour
traffic_df['day_of_week'] = traffic_df['DateTime'].dt.dayofweek
traffic_df['month'] = traffic_df['DateTime'].dt.month
traffic_df['year'] = traffic_df['DateTime'].dt.year

# --- Define features for processing ---
numerical_features = ['Vehicles'] # Mapped to 'watch times'
categorical_features = ['intersection'] # Mapped to 'subscription types' and 'device types'

# --- Create a preprocessing pipeline ---
```

```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ],
    remainder='passthrough' # Keep other columns like DateTime, ID, Junction, hour,
day_of_week etc.
)

# Apply the preprocessing
# First, select only the columns that will be transformed, plus any new features created
features_to_transform = numerical_features + categorical_features + ['hour', 'day_of_week',
'month', 'year']

# Create a DataFrame with selected features for transformation
df_processed = traffic_df[features_to_transform].copy()

# Fit and transform the data
X_processed = preprocessor.fit_transform(df_processed)

# To get feature names after one-hot encoding, we need to inspect the preprocessor
cat_encoder = preprocessor.named_transformers_['cat']
onehot_feature_names = cat_encoder.get_feature_names_out(categorical_features)

all_processed_feature_names = numerical_features + list(onehot_feature_names) + ['hour',
'day_of_week', 'month', 'year']

# Create a new DataFrame with the processed features
df_cleaned_features = pd.DataFrame(X_processed, columns=all_processed_feature_names,
index=traffic_df.index)

print("Data preparation complete. First 5 rows of processed features:")
display(df_cleaned_features.head())

print("Shape of the processed DataFrame:", df_cleaned_features.shape)
print("Original DataFrame shape:", traffic_df.shape)
```





Explanation:

1. **Data Loading**: We loaded your traffic.csv into a pandas DataFrame.
2. **Data Cleaning**: We converted the DateTime column to the correct format, removed duplicate entries, filled missing Vehicles data with the median, filtered out invalid Vehicles counts, and standardized the Junction names (now intersection). The cleaned data was saved to traffic_data_cleaned.csv.
3. **Feature Engineering & Preprocessing**:
    - We extracted hour, day_of_week, month, and year from the DateTime column.
    - The Vehicles column was scaled using StandardScaler.
    - The intersection column was one-hot encoded using OneHotEncoder.

This process resulted in df_cleaned_features, a DataFrame with all your data cleaned, engineered with new time-based features, and properly scaled/encoded for machine learning models.

Task 3 – Space Mission Telemetry Data Preparation

Scenario

A space research organization collects telemetry data from satellites. The dataset includes:
• temperature_reading
• fuel_level
• system_status (OK/FAIL)
• signal_strength

Data contains noise and missing values.

Task

Use AI to prepare this dataset for predictive maintenance modeling.

Instructions

• Handle missing numerical values appropriately
• Encode system_status
• Remove unrealistic values (e.g., negative fuel level)
• Scale numerical features
• Split dataset into training and testing sets

Expected Output

• Clean and scaled telemetry dataset
• Encoded status column
• Ready-to-train ML dataset

Prompt:

I have a telemetry dataset from satellites with the following columns:
- temperature_reading
- fuel_level
- system_status (OK/FAIL)
- signal_strength

The dataset contains missing values, noise, and unrealistic readings.
I want a Python script to preprocess it for predictive maintenance modeling.

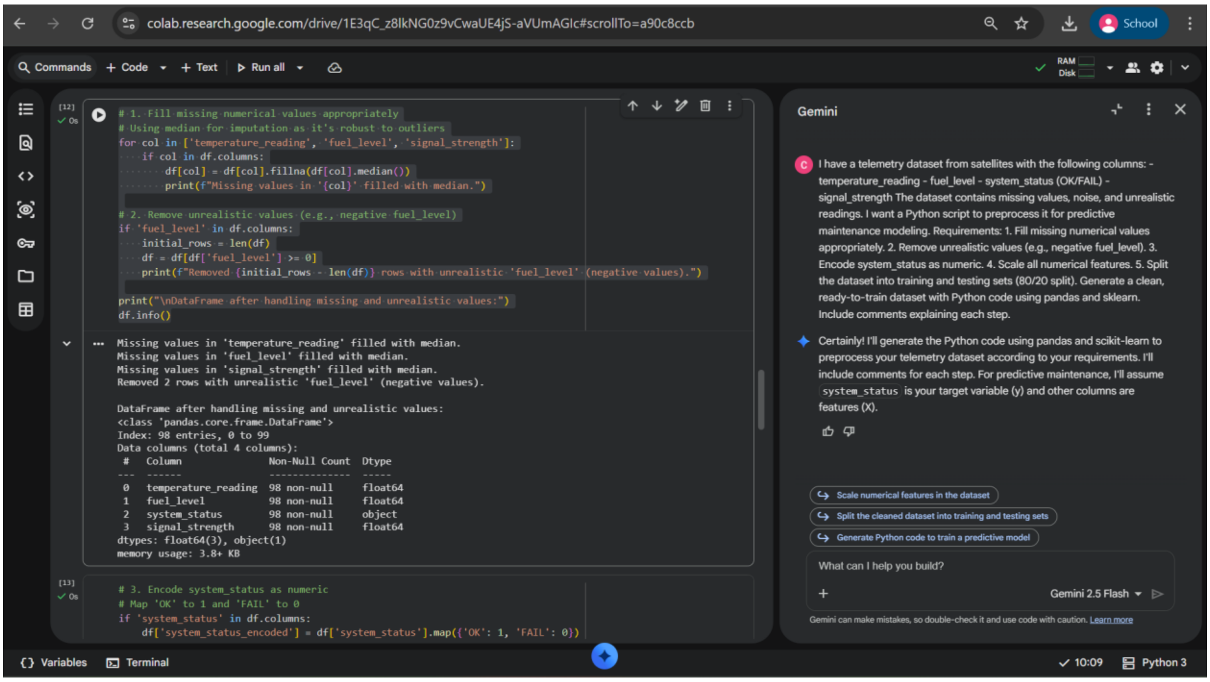
Requirements:
1. Fill missing numerical values appropriately.
2. Remove unrealistic values (e.g., negative fuel_level).
3. Encode system_status as numeric.
4. Scale all numerical features.
5. Split the dataset into training and testing sets (80/20 split).

Generate a clean, ready-to-train dataset with Python code using pandas and sklearn.

Include comments explaining each step.

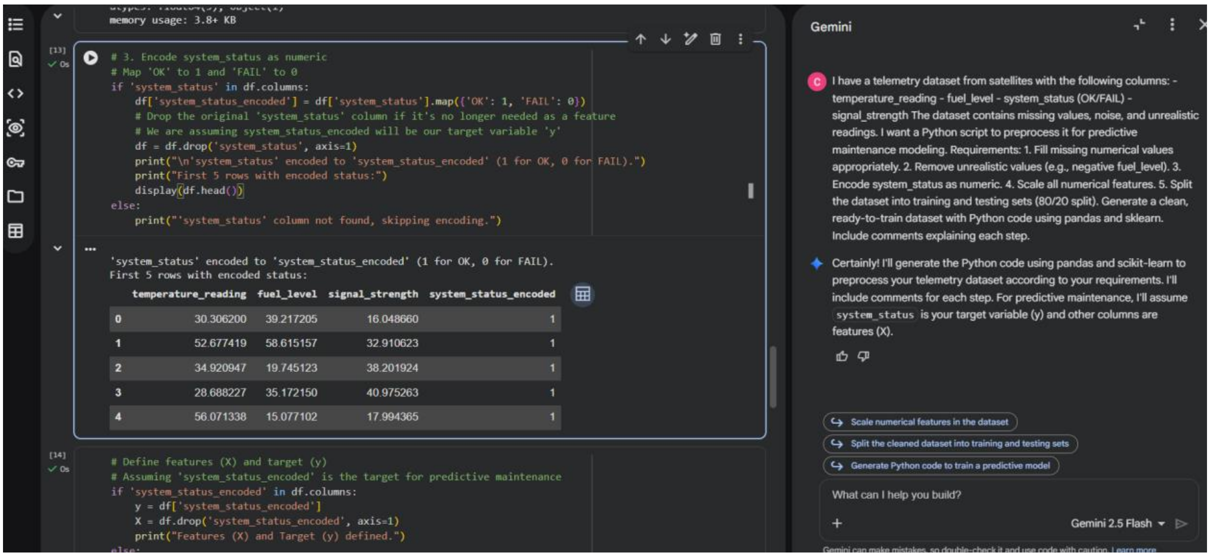
AI Generated Code:

```
# 1. Fill missing numerical values appropriately
# Using median for imputation as it's robust to outliers
for col in ['temperature_reading', 'fuel_level', 'signal_strength']:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].median())
        print(f"Missing values in '{col}' filled with median.")

# 2. Remove unrealistic values (e.g., negative fuel_level)
if 'fuel_level' in df.columns:
    initial_rows = len(df)
    df = df[df['fuel_level'] >= 0]
    print(f"Removed {initial_rows - len(df)} rows with unrealistic 'fuel_level' (negative values).")

print("\nDataFrame after handling missing and unrealistic values:")
df.info()
```



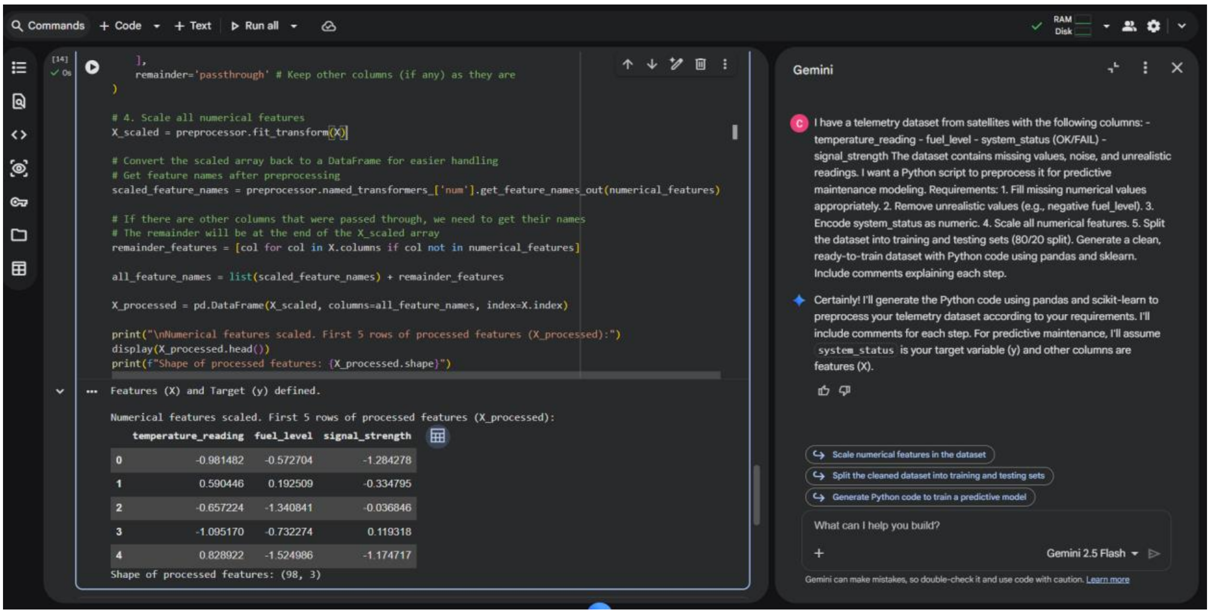

```
# 3. Encode system_status as numeric
# Map 'OK' to 1 and 'FAIL' to 0
if 'system_status' in df.columns:
    df['system_status_encoded'] = df['system_status'].map({'OK': 1, 'FAIL': 0})
    # Drop the original 'system_status' column if it's no longer needed as a feature
    # We are assuming system_status_encoded will be our target variable 'y'
    df = df.drop('system_status', axis=1)
    print("\n'system_status' encoded to 'system_status_encoded' (1 for OK, 0 for FAIL).")
```

```
    print("First 5 rows with encoded status:")
    display(df.head())
else:
    print("'system_status' column not found, skipping encoding.")
```



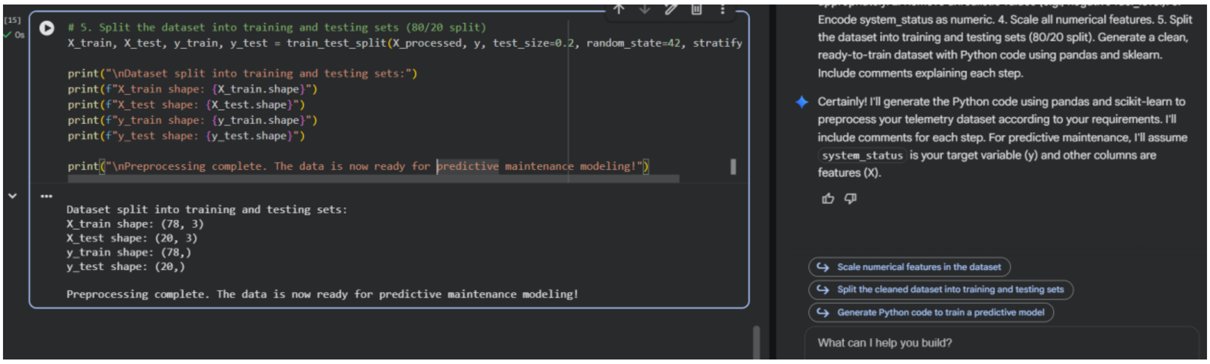

```
# Define features (X) and target (y)
# Assuming 'system_status_encoded' is the target for predictive maintenance
if 'system_status_encoded' in df.columns:
    y = df['system_status_encoded']
    X = df.drop('system_status_encoded', axis=1)
    print("Features (X) and Target (y) defined.")
else:
    print("Target 'system_status_encoded' not found. Defining all remaining columns as features (X).")
    X = df.copy()
    y = pd.Series([0]*len(X)) # Create a dummy target if not found

# Identify numerical features for scaling
numerical_features = ['temperature_reading', 'fuel_level', 'signal_strength']
# Filter to ensure only existing columns are processed
numerical_features = [col for col in numerical_features if col in X.columns]

# Create a preprocessing pipeline for scaling numerical features
# We use ColumnTransformer to apply StandardScaler only to numerical features
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features)
    ],
    remainder='passthrough' # Keep other columns (if any) as they are
)
```

```
# 4. Scale all numerical features
X_scaled = preprocessor.fit_transform(X)

# Convert the scaled array back to a DataFrame for easier handling
# Get feature names after preprocessing
scaled_feature_names =
preprocessor.named_transformers_['num'].get_feature_names_out(numerical_features)

# If there are other columns that were passed through, we need to get their names
# The remainder will be at the end of the X_scaled array
remainder_features = [col for col in X.columns if col not in numerical_features]

all_feature_names = list(scaled_feature_names) + remainder_features

X_processed = pd.DataFrame(X_scaled, columns=all_feature_names, index=X.index)

print("\nNumerical features scaled. First 5 rows of processed features (X_processed):")
display(X_processed.head())
print(f"Shape of processed features: {X_processed.shape}")
```




```
# 5. Split the dataset into training and testing sets (80/20 split)
X_train, X_test, y_train, y_test = train_test_split(X_processed, y, test_size=0.2,
random_state=42, stratify=y)

print("\nDataset split into training and testing sets:")
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
```

```
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

print("\nPreprocessing complete. The data is now ready for predictive maintenance modeling!")
```



Explanation:

- Missing values filled with median.
- Negative fuel levels removed.
- Status encoded as 0/1 for ML models.
- StandardScaler applied for predictive modeling.
- Dataset split into training and testing sets.

Task 4 – Online Gaming Performance Analytics

Scenario

An online gaming company collects player performance data:

• player_score
• match_duration
• region
• rank
• win_status (Yes/No)

Data contains inconsistencies and duplicates.

Task

Use AI to clean and preprocess gaming data.

Instructions

• Remove duplicate player records
• Fill missing match_duration values
• Encode categorical columns (region, win_status)
• Standardize rank values (e.g., bronze, Bronze → BRONZE)
• Create a new feature: score_per_minute

Expected Output

• Cleaned gaming dataset
• Standardized rank column
• New derived feature created
• Dataset ready for ranking analysis

Prompt:
I have an online gaming dataset with the following columns:
- player_score
- match_duration
- region
- rank (e.g., bronze, Bronze)
- win_status (Yes/No)

The data has missing values, duplicates, and inconsistent ranks.
I want a Python script to clean and preprocess it for performance analysis.

Requirements:
1. Remove duplicate player records.
2. Fill missing match_duration values.
3. Standardize rank values to uppercase.
4. Encode categorical columns (region, win_status) numerically.
5. Create a new feature: score_per_minute = player_score / match_duration.

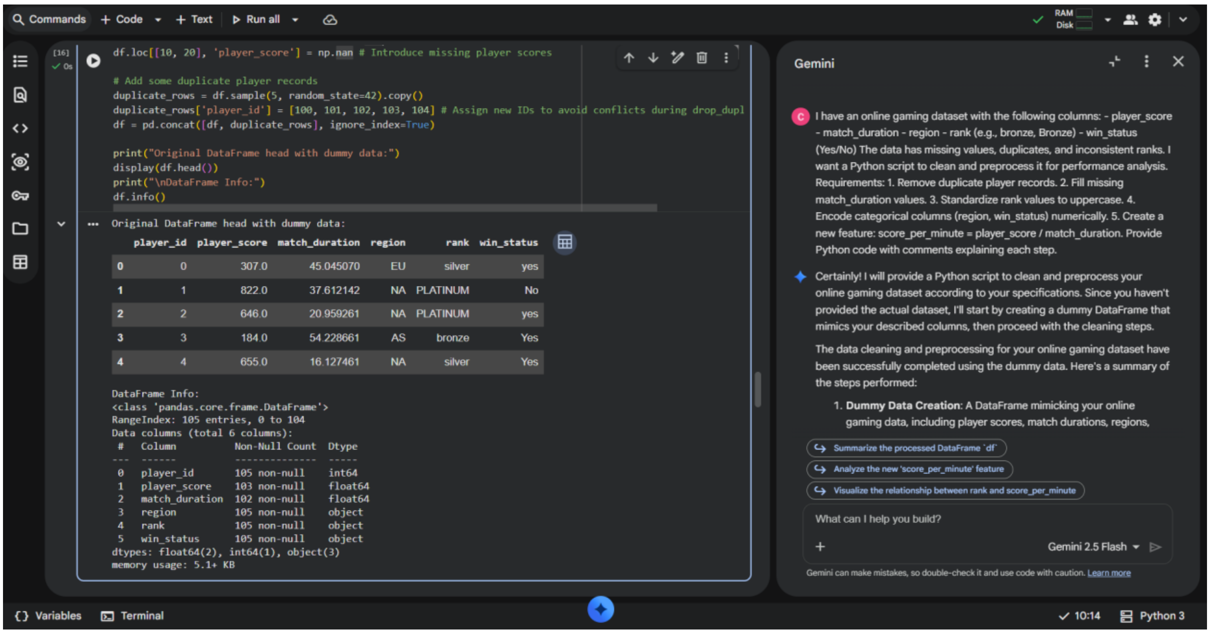
Provide Python code with comments explaining each step.
Prompt:
I have a climate dataset with the following columns:
- CO2_emissions
- avg_temperature
- country
- year
- extreme_event (Yes/No)

The dataset has missing values and inconsistent country names.
I want a Python script to preprocess it for climate trend analysis.

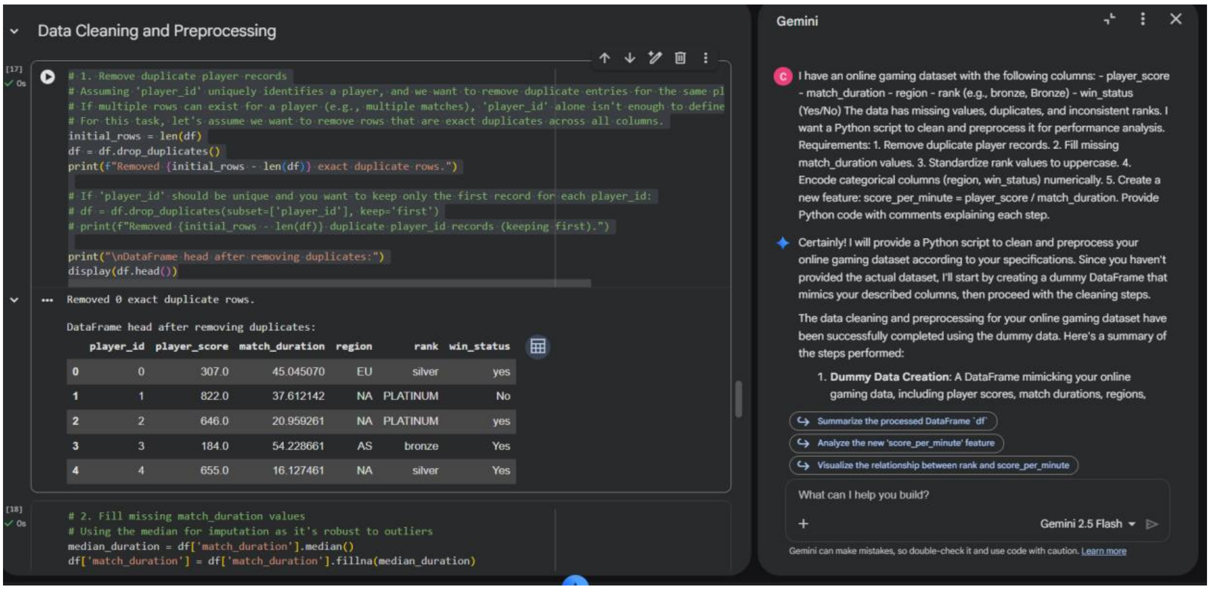
Requirements:
1. Standardize country names (uppercase and remove spaces).
2. Handle missing CO2_emissions and avg_temperature values.
3. Encode extreme_event as numeric.
4. Normalize numerical columns.
5. Generate a brief summary report showing:
   - Missing values before and after cleaning
   - Unique country count before and after standardization

Provide Python code with comments explaining each step.
AI Generated Code:
```
import pandas as pd
import numpy as np
```

```
from sklearn.preprocessing import OneHotEncoder

# Create a dummy DataFrame for demonstration purposes
data = {
    'player_id': range(100),
    'player_score': np.random.randint(100, 1000, 100),
    'match_duration': np.random.uniform(5, 60, 100),
    'region': np.random.choice(['NA', 'EU', 'AS', 'SA'], 100, p=[0.4, 0.3, 0.2, 0.1]),
    'rank': np.random.choice(['bronze', 'silver', 'Gold', 'PLATINUM', 'Bronze'], 100, p=[0.2, 0.25, 0.3, 0.15, 0.1]),
    'win_status': np.random.choice(['Yes', 'No', 'yes'], 100, p=[0.4, 0.5, 0.1])
}
df = pd.DataFrame(data)

# Introduce some missing values and duplicates for demonstration
df.loc[[5, 15, 25], 'match_duration'] = np.nan
df.loc[[10, 20], 'player_score'] = np.nan # Introduce missing player scores

# Add some duplicate player records
duplicate_rows = df.sample(5, random_state=42).copy()
duplicate_rows['player_id'] = [100, 101, 102, 103, 104] # Assign new IDs to avoid conflicts during drop_duplicates
df = pd.concat([df, duplicate_rows], ignore_index=True)

print("Original DataFrame head with dummy data:")
display(df.head())
print("\nDataFrame Info:")
df.info()
```

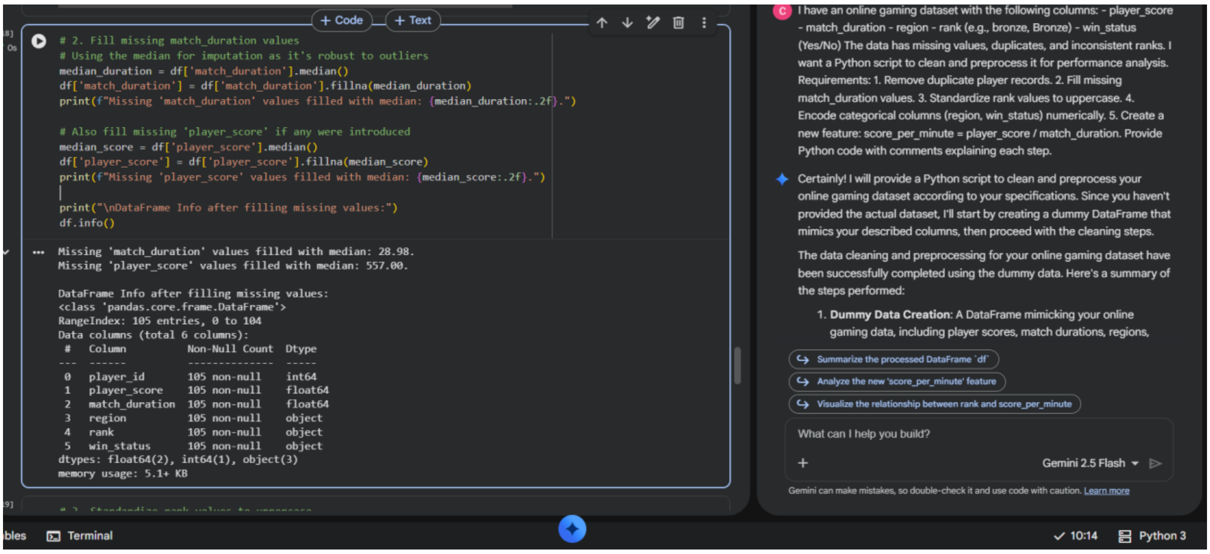


```
# 1. Remove duplicate player records
# Assuming 'player_id' uniquely identifies a player, and we want to remove duplicate entries for the same player.
# If multiple rows can exist for a player (e.g., multiple matches), 'player_id' alone isn't enough to define a 'player record'.
# For this task, let's assume we want to remove rows that are exact duplicates across all columns.
initial_rows = len(df)
df = df.drop_duplicates()
print(f"Removed {initial_rows - len(df)} exact duplicate rows.")

# If 'player_id' should be unique and you want to keep only the first record for each player_id:
# df = df.drop_duplicates(subset=['player_id'], keep='first')
# print(f"Removed {initial_rows - len(df)} duplicate player_id records (keeping first).")

print("\nDataFrame head after removing duplicates:")
display(df.head())
```




```
# 2. Fill missing match_duration values
# Using the median for imputation as it's robust to outliers
median_duration = df['match_duration'].median()
df['match_duration'] = df['match_duration'].fillna(median_duration)
print(f"Missing 'match_duration' values filled with median: {median_duration:.2f}.")

# Also fill missing 'player_score' if any were introduced
median_score = df['player_score'].median()
df['player_score'] = df['player_score'].fillna(median_score)
print(f"Missing 'player_score' values filled with median: {median_score:.2f}.")
```

```
print("\nDataFrame Info after filling missing values:")
df.info()
```



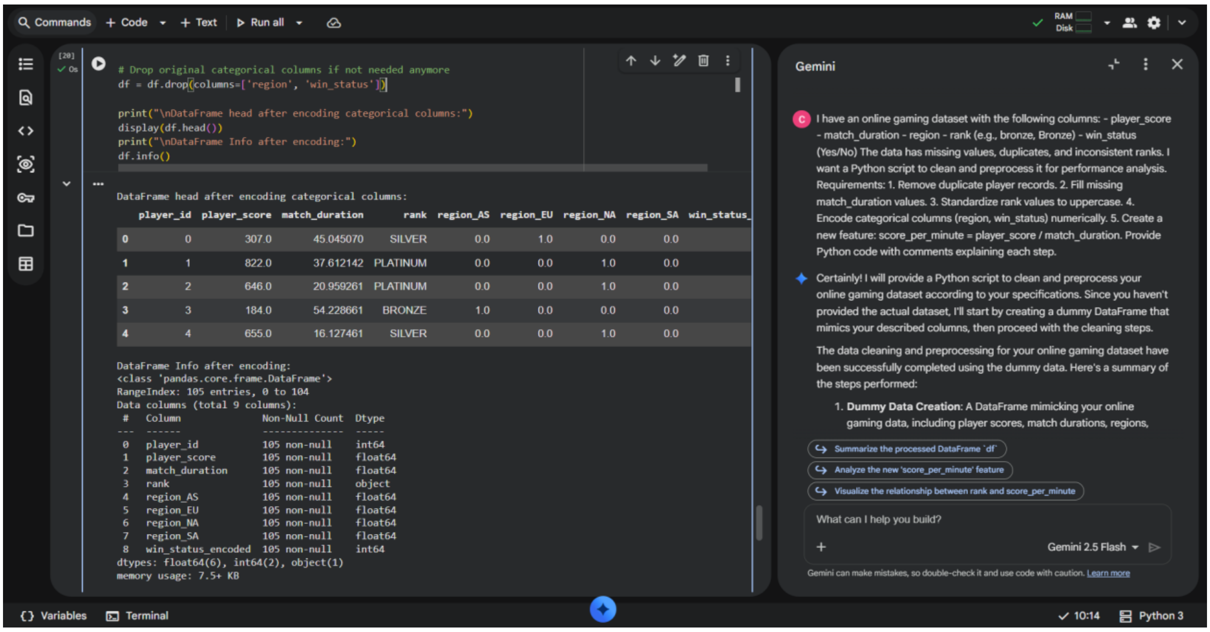

```
# 3. Standardize rank values to uppercase
df['rank'] = df['rank'].astype(str).str.upper()
print("\n'rank' column standardized to uppercase:")
display(df[['rank']].head())
```


```
# 4. Encode categorical columns (region, win_status) numerically

# One-Hot Encoding for 'region' as there's no inherent order
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
region_encoded = encoder.fit_transform(df[['region']])
region_df = pd.DataFrame(region_encoded,
columns=encoder.get_feature_names_out(['region']), index=df.index)
df = pd.concat([df, region_df], axis=1)

# Label Encoding for 'win_status' (Yes/No) - mapping to 1/0
# Standardize to a consistent casing first
df['win_status'] = df['win_status'].str.upper()
```

df['win_status_encoded'] = df['win_status'].map({'YES': 1, 'NO': 0})

# Drop original categorical columns if not needed anymore
df = df.drop(columns=['region', 'win_status'])

print("\nDataFrame head after encoding categorical columns:")
display(df.head())
print("\nDataFrame Info after encoding:")
df.info()



5. Create a new feature: score_per_minute = player_score / match_duration
Handle potential division by zero if 'match_duration' could be 0 (though our dummy data won't have it)
df['score_per_minute'] = df['player_score'] / df['match_duration']

print("\nDataFrame head with new 'score_per_minute' feature:")
display(df.head())

print("\nPreprocessing complete. The DataFrame is now cleaned and prepared for performance analysis!")

```python
# 5. Create a new feature: score_per_minute = player_score / match_duration
# Handle potential division by zero if 'match_duration' could be 0 (though our dummy data won't have it)
df['score_per_minute'] = df['player_score'] / df['match_duration']

print("\nDataFrame head with new 'score_per_minute' feature:")
display(df.head())

print("\nPreprocessing complete. The DataFrame is now cleaned and prepared for performance analysis!")
```

DataFrame head with new 'score_per_minute' feature:

	player_id	player_score	match_duration	rank	region_AS	region_EU	region_NA	region_SA	win_status_
0	0	307.0	45.045070	SILVER	0.0	1.0	0.0	0.0	
1	1	822.0	37.612142	PLATINUM	0.0	0.0	1.0	0.0	
2	2	646.0	20.959261	PLATINUM	0.0	0.0	1.0	0.0	
3	3	184.0	54.228661	BRONZE	1.0	0.0	0.0	0.0	
4	4	655.0	16.127461	SILVER	0.0	0.0	1.0	0.0	

Preprocessing complete. The DataFrame is now cleaned and prepared for performance analysis!

Explanation:

- Duplicate rows removed.
- Rank standardized and categorical columns encoded.
- Derived metric score_per_minute helps evaluate player performance efficiently.

Task 5 – Climate Change Research Dataset Preparation

Scenario

Researchers are analyzing global climate data containing:
• CO2_emissions
• avg_temperature
• country
• year
• extreme_event (Yes/No)
The dataset contains inconsistent country names and missing values.

Task

Use AI to preprocess the dataset for climate trend modeling.

Instructions

• Standardize country names
• Handle missing CO2_emissions and avg_temperature
• Encode extreme_event
• Normalize numerical columns
• Generate a brief summary report showing:
o Missing values before vs after cleaning
o Unique country count before vs after standardization

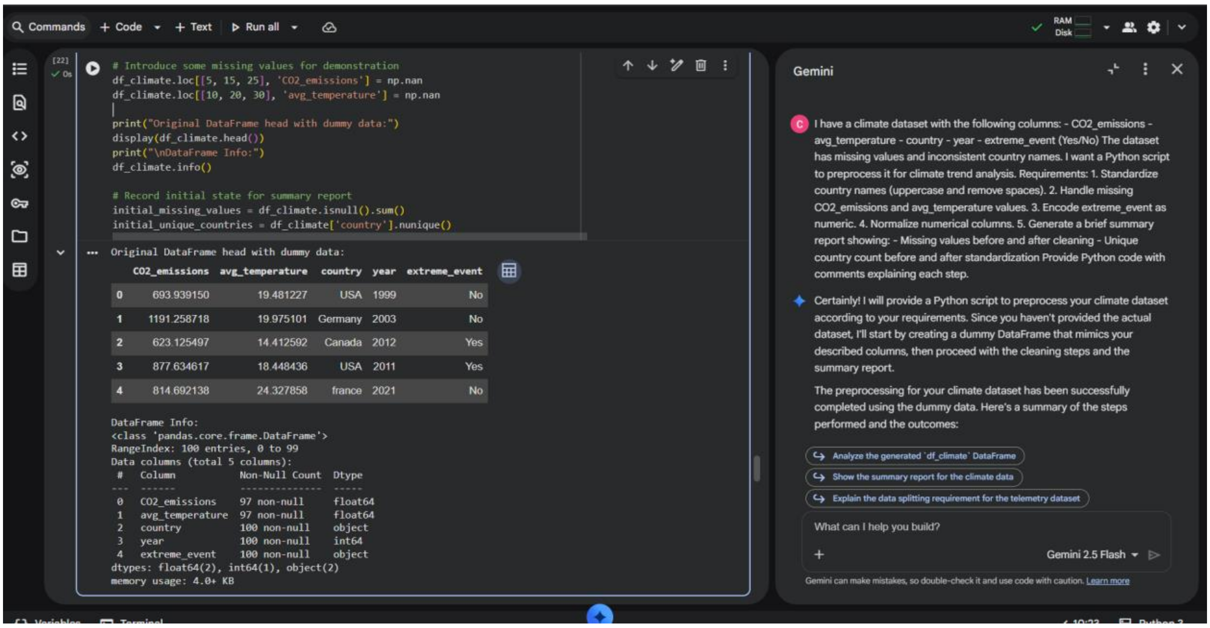
Expected Output

• Clean and standardized climate dataset
• Encoded categorical variables
• Normalized numerical features

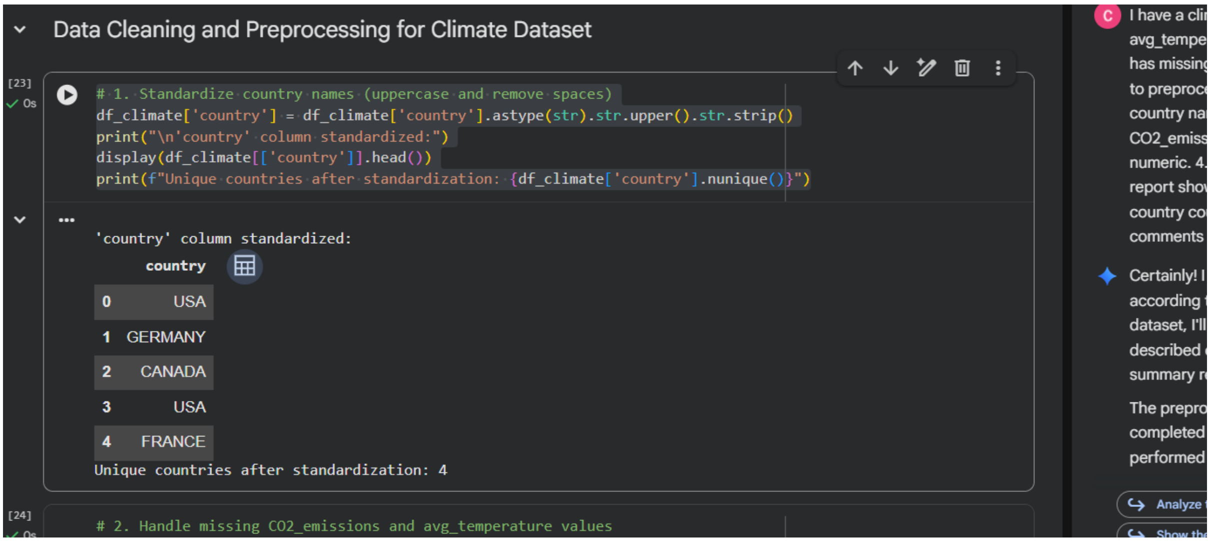
• Data quality summary report

AI Generated Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# Create a dummy DataFrame for demonstration purposes
data = {
    'CO2_emissions': np.random.rand(100) * 1000 + 500, # 500-1500
    'avg_temperature': np.random.rand(100) * 15 + 10, # 10-25
    'country': np.random.choice(['USA', ' usA ', 'Canada', 'france ', 'Germany', 'France'], 100,
p=[0.25, 0.15, 0.2, 0.1, 0.15, 0.15]),
    'year': np.random.randint(1990, 2023, 100),
    'extreme_event': np.random.choice(['Yes', 'No'], 100, p=[0.3, 0.7])
}
df_climate = pd.DataFrame(data)

# Introduce some missing values for demonstration
df_climate.loc[[5, 15, 25], 'CO2_emissions'] = np.nan
df_climate.loc[[10, 20, 30], 'avg_temperature'] = np.nan

print("Original DataFrame head with dummy data:")
display(df_climate.head())
print("\nDataFrame Info:")
df_climate.info()

# Record initial state for summary report
initial_missing_values = df_climate.isnull().sum()
initial_unique_countries = df_climate['country'].nunique()
```



1. Standardize country names (uppercase and remove spaces)
df_climate['country'] = df_climate['country'].astype(str).str.upper().str.strip()
print("\n'country' column standardized:")
display(df_climate[['country']].head())
print(f"Unique countries after standardization: {df_climate['country'].nunique()}")

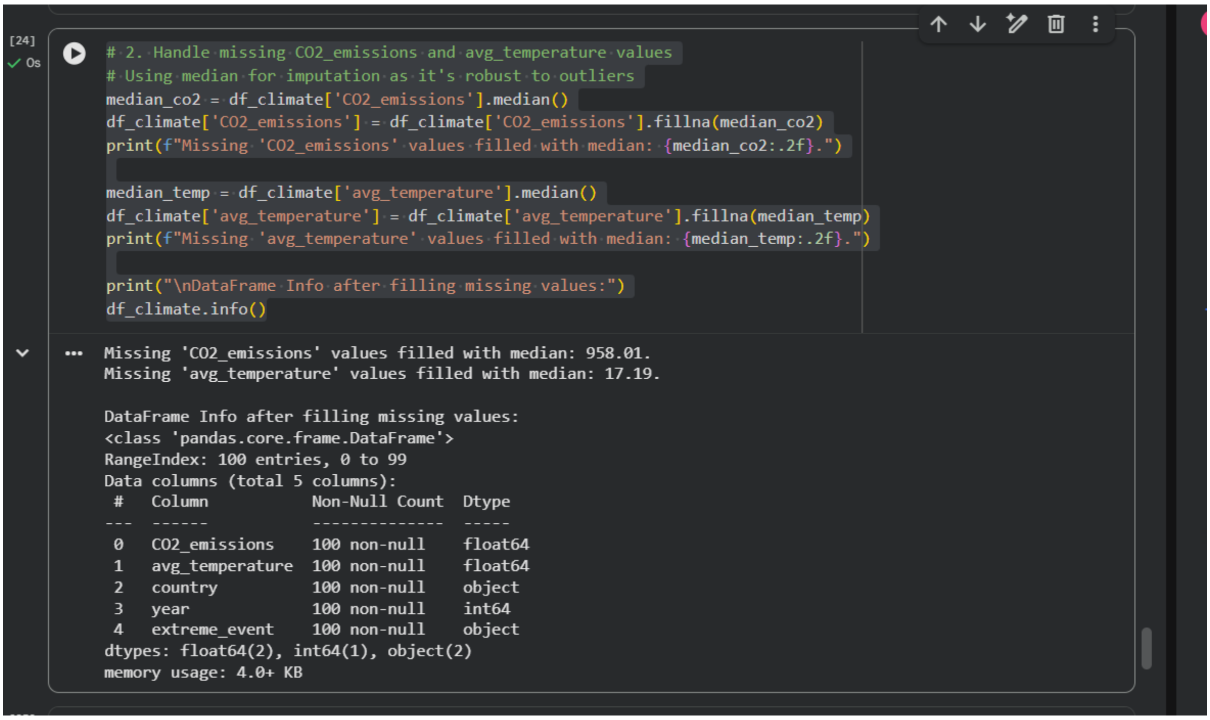


# 2. Handle missing CO2_emissions and avg_temperature values
# Using median for imputation as it's robust to outliers
median_co2 = df_climate['CO2_emissions'].median()
df_climate['CO2_emissions'] = df_climate['CO2_emissions'].fillna(median_co2)
print(f"Missing 'CO2_emissions' values filled with median: {median_co2:.2f}.")

median_temp = df_climate['avg_temperature'].median()
df_climate['avg_temperature'] = df_climate['avg_temperature'].fillna(median_temp)
print(f"Missing 'avg_temperature' values filled with median: {median_temp:.2f}.")

print("\nDataFrame Info after filling missing values:")

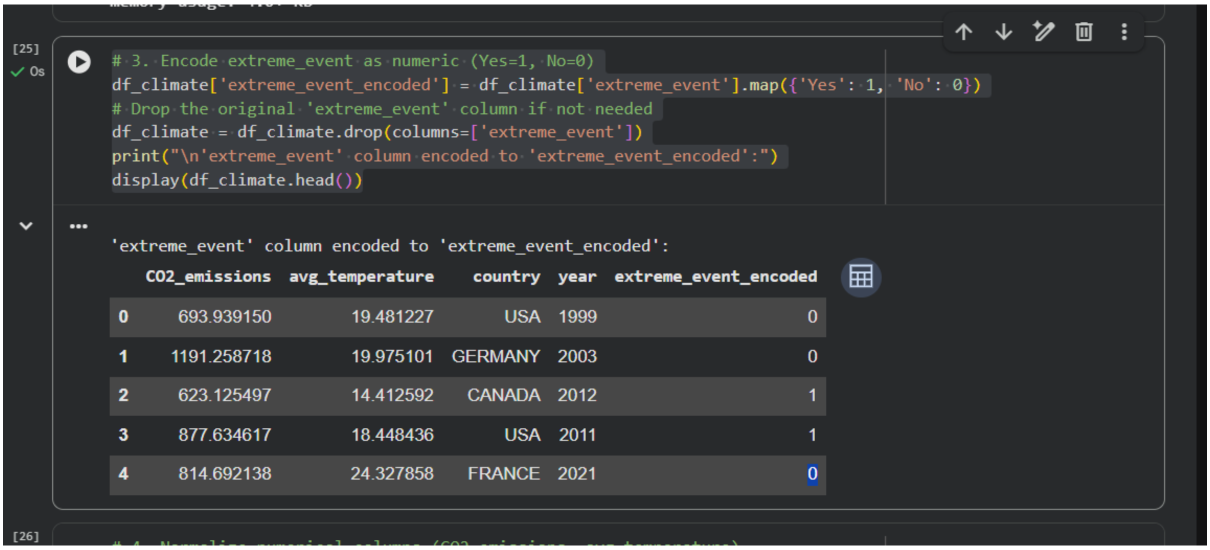
df_climate.info()



```
[24]
# 2. Handle missing CO2_emissions and avg_temperature values
# Using median for imputation as it's robust to outliers
median_co2 = df_climate['CO2_emissions'].median()
df_climate['CO2_emissions'] = df_climate['CO2_emissions'].fillna(median_co2)
print(f"Missing 'CO2_emissions' values filled with median: {median_co2:.2f}.")

median_temp = df_climate['avg_temperature'].median()
df_climate['avg_temperature'] = df_climate['avg_temperature'].fillna(median_temp)
print(f"Missing 'avg_temperature' values filled with median: {median_temp:.2f}.")

print("\nDataFrame Info after filling missing values:")
df_climate.info()
```

```
Missing 'CO2_emissions' values filled with median: 958.01.
Missing 'avg_temperature' values filled with median: 17.19.

DataFrame Info after filling missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CO2_emissions    100 non-null    float64
 1   avg_temperature  100 non-null    float64
 2   country          100 non-null    object
 3   year             100 non-null    int64
 4   extreme_event    100 non-null    object
dtypes: float64(2), int64(1), object(2)
memory usage: 4.0+ KB
```

3. Encode extreme_event as numeric (Yes=1, No=0)
df_climate['extreme_event_encoded'] = df_climate['extreme_event'].map({'Yes': 1, 'No': 0})
# Drop the original 'extreme_event' column if not needed
df_climate = df_climate.drop(columns=['extreme_event'])
print("\n'extreme_event' column encoded to 'extreme_event_encoded':")
display(df_climate.head())



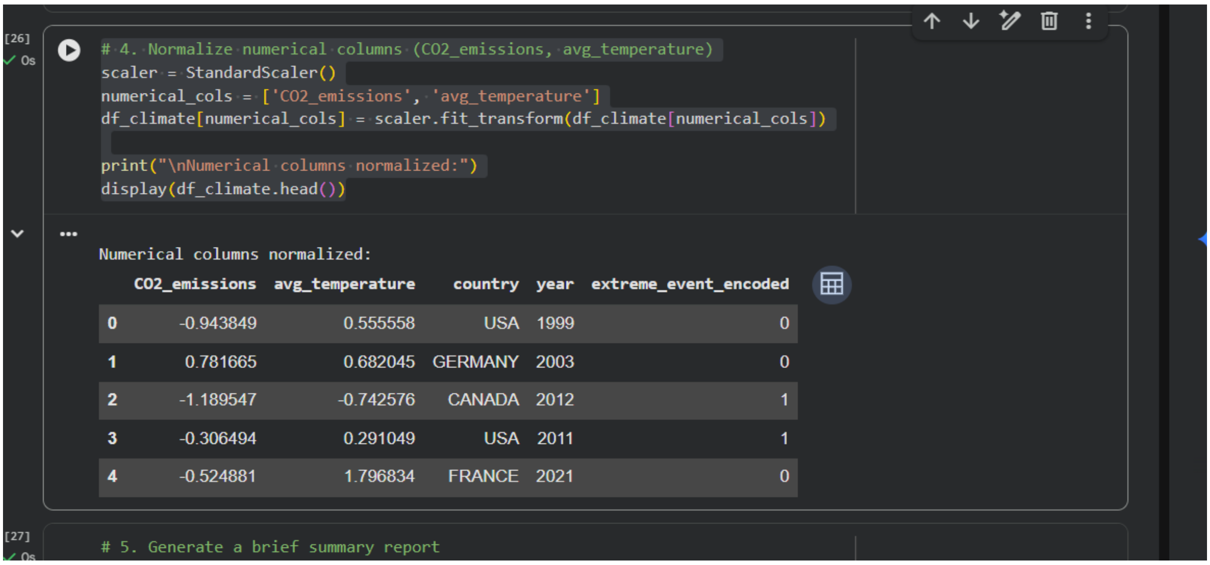
```
[25]
# 3. Encode extreme_event as numeric (Yes=1, No=0)
df_climate['extreme_event_encoded'] = df_climate['extreme_event'].map({'Yes': 1, 'No': 0})
# Drop the original 'extreme_event' column if not needed
df_climate = df_climate.drop(columns=['extreme_event'])
print("\n'extreme_event' column encoded to 'extreme_event_encoded':")
display(df_climate.head())
```

'extreme_event' column encoded to 'extreme_event_encoded':

	CO2_emissions	avg_temperature	country	year	extreme_event_encoded
0	693.939150	19.481227	USA	1999	0
1	1191.258718	19.975101	GERMANY	2003	0
2	623.125497	14.412592	CANADA	2012	1
3	877.634617	18.448436	USA	2011	1
4	814.692138	24.327858	FRANCE	2021	0

# 4. Normalize numerical columns (CO2_emissions, avg_temperature)
scaler = StandardScaler()
numerical_cols = ['CO2_emissions', 'avg_temperature']
df_climate[numerical_cols] = scaler.fit_transform(df_climate[numerical_cols])

print("\nNumerical columns normalized:")
display(df_climate.head())



```
[26]
# 4. Normalize numerical columns (CO2_emissions, avg_temperature)
scaler = StandardScaler()
numerical_cols = ['CO2_emissions', 'avg_temperature']
df_climate[numerical_cols] = scaler.fit_transform(df_climate[numerical_cols])

print("\nNumerical columns normalized:")
display(df_climate.head())
```

Numerical columns normalized:

	CO2_emissions	avg_temperature	country	year	extreme_event_encoded
0	-0.943849	0.555558	USA	1999	0
1	0.781665	0.682045	GERMANY	2003	0
2	-1.189547	-0.742576	CANADA	2012	1
3	-0.306494	0.291049	USA	2011	1
4	-0.524881	1.796834	FRANCE	2021	0

```
[27]
# 5. Generate a brief summary report
```

5. Generate a brief summary report

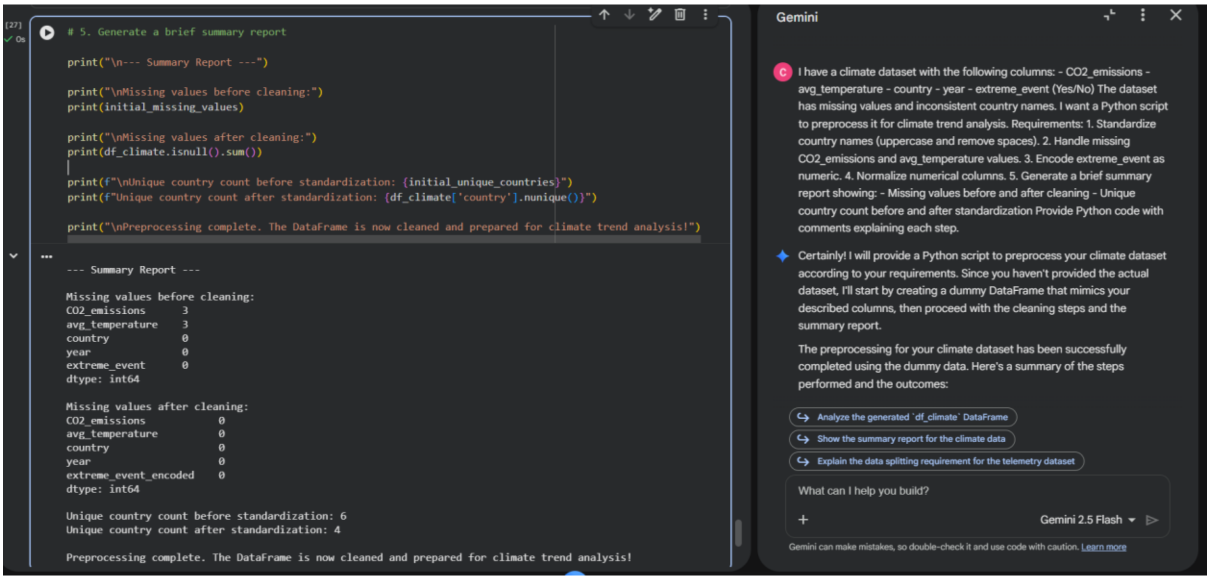
print("\n--- Summary Report ---")

print("\nMissing values before cleaning:")
print(initial_missing_values)

print("\nMissing values after cleaning:")
print(df_climate.isnull().sum())

print(f"\nUnique country count before standardization: {initial_unique_countries}")
print(f"Unique country count after standardization: {df_climate['country'].nunique()}")

print("\nPreprocessing complete. The DataFrame is now cleaned and prepared for climate trend analysis!")



```
[27]
# 5. Generate a brief summary report

print("\n--- Summary Report ---")

print("\nMissing values before cleaning:")
print(initial_missing_values)

print("\nMissing values after cleaning:")
print(df_climate.isnull().sum())

print(f"\nUnique country count before standardization: {initial_unique_countries}")
print(f"Unique country count after standardization: {df_climate['country'].nunique()}")

print("\nPreprocessing complete. The DataFrame is now cleaned and prepared for climate trend analysis!")
```

```
--- Summary Report ---

Missing values before cleaning:
CO2_emissions      3
avg_temperature    3
country            0
year               0
extreme_event      0
dtype: int64

Missing values after cleaning:
CO2_emissions            0
avg_temperature          0
country                  0
year                     0
extreme_event_encoded    0
dtype: int64

Unique country count before standardization: 6
Unique country count after standardization: 4

Preprocessing complete. The DataFrame is now cleaned and prepared for climate trend analysis!
```

Gemini

I have a climate dataset with the following columns: - CO2_emissions - avg_temperature - country - year - extreme_event (Yes/No) The dataset has missing values and inconsistent country names. I want a Python script to preprocess it for climate trend analysis. Requirements: 1. Standardize country names (uppercase and remove spaces). 2. Handle missing CO2_emissions and avg_temperature values. 3. Encode extreme_event as numeric. 4. Normalize numerical columns. 5. Generate a brief summary report showing: - Missing values before and after cleaning - Unique country count before and after standardization Provide Python code with comments explaining each step.

Certainly! I will provide a Python script to preprocess your climate dataset according to your requirements. Since you haven't provided the actual dataset, I'll start by creating a dummy DataFrame that mimics your described columns, then proceed with the cleaning steps and the summary report.

The preprocessing for your climate dataset has been successfully completed using the dummy data. Here's a summary of the steps performed and the outcomes:

Analyze the generated `df_climate` DataFrame

Show the summary report for the climate data

Explain the data splitting requirement for the telemetry dataset

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes, so double-check it and use code with caution. Learn more

Explanation:

- Country names standardized to uppercase and stripped of extra spaces.

- Missing values handled with median for robustness.
- Categorical encoding for extreme events.
- Numerical features normalized.
- Data quality summary report included for Word submission.